异步派生记忆的可验证删除：基于 SQLite 的定序消融与故障恢复研究

Verifiable Deletion of Asynchronously Derived Memory: A SQLite Study with Controlled Schedules and Recovery Probes

研究论文 · Aervox-harness · 2026 年 10 月 15 日

摘要

持久记忆系统中的删除不能只依据主记录的删除标记判定。全文索引、向量、压缩摘要、审计文本和待处理事件可能继续保留来源内容；删除前已读取数据的工作者还可能在清理后将其写回。本文研究本地单用户 SQLite 系统中一个有明确边界的 Memory 删除切片，将删除完成拆为来源失效、关联内容清理、迟到写入拒绝和独立完成验证四项条件。我们在真实仓储组件上构建四个组件消融臂，以五类派生通道、三个写入位置和五种清理遗漏形成每臂 20 个固定案例，另测量每臂 24 条操作耗时，并设置错误回写与进程终止探针。

80 个正确性案例中，仅设置删除标记的消融臂有 20 次错误完成；只清理的消融臂有 10 次错误完成和 5 次完成后复活；增加写入守卫后，复活降为 0，但仍有 5 次遗漏导致的错误完成。清理、守卫和验证共同启用时，15 个无遗漏案例安全完成，5 个遗漏案例被正确阻塞，没有观测到错误完成或完成后复活。研究同时发现并修复了生产 Outbox 的迟到错误文本回写，以及删除验证器对扩展载荷和墓碑状态的漏检。两处 SIGKILL 探针在重开与重试后均恢复到所定义的干净状态。结果支持将写入守卫与独立验证作为互补机制；它不构成任意并发执行下的证明，也不等价于模型语义遗忘、备份删除或物理介质擦除。

关键词：持久记忆；数据删除；SQLite；异步工作者；故障注入；可复现研究

Abstract

Deleting a persistent memory record does not by itself remove derived content or prevent a worker from writing previously fetched information back to storage. We study a bounded Memory deletion slice in a local, single-user SQLite application. The mechanism combines source invalidation, cleanup of associated content, guarded writes, and a fresh completion check. Four component ablations are evaluated on 20 predetermined cases each, with five derivative channels, three write positions, and five cleanup omissions. We separately collect 24 timing samples per arm, compare legacy and patched error-write paths, and probe two process-termination boundaries.

Soft deletion produces 20 incorrect completions. Cleanup alone produces 10 incorrect completions and five later resurrections. Guarded cleanup prevents the observed resurrections but still declares completion in five omission cases. Adding verification yields 15 safe completions and five correctly blocked cases, with no incorrect completion or resurrection in the selected matrix. The study also identifies and fixes late Outbox error writes and incomplete tombstone verification. Both selected SIGKILL probes recover after reopening and retrying. These results establish bounded implementation evidence for complementary write guards and completion checks, rather than general concurrency correctness, semantic unlearning, backup erasure, or secure physical deletion.

1. 引言

面向长期交互的应用常将一条用户事实转换为多个可持久化对象。原文进入主记录与历史修订，检索系统建立全文索引和向量，后台工作者生成摘要或压缩标记，事件系统保留调度载荷与失败原因。用户发出删除请求时，这些对象不一定与主记录处于同一生命周期。尤其是在后台任务已经获取正文、随后删除完成的情况下，任务的正常结果或错误消息都可能成为重新写入内容的途径。

这种问题包含两个不同的时间条件。第一，宣告完成时，受本次删除范围约束的逻辑存储中是否仍有内容残留；第二，完成后的迟到写入能否使残留重新出现。检查清理语句的影响行数无法充分回答前者，在方法入口做一次存活检查也无法充分回答后者。检查与写入之间仍可能发生删除，错误处理分支也可能绕过正常结果路径上的守卫。

本文围绕三个问题展开。RQ1：来源标记、内容清理、写入守卫和完成验证分别消除哪些失败模式？RQ2：失败处理路径是否会绕过清理，且能否用原子更新消除所观察的窗口？RQ3：所选未提交和已提交边界遭进程终止后，数据库状态及幂等重试是否符合预期？耗时实验提供这些机制在一个小型合成夹具上的描述性成本，不承担吞吐量或规模扩展性结论。

本文的贡献是一个可复跑的定序消融实验、一个与生产验证器实现分离的残留观察器，以及实验驱动的两类生产修复。实验保存逐案例观察、计时样本、JSON/CSV、关键源文件与构建产物的哈希和绘图脚本。所有正确性结论均限定在声明的对象范围、调度与故障边界内。

2. 相关工作与定位

数据库来源研究区分结果为什么存在及其值来自何处，Buneman、Khanna 和 Tan 的工作为追踪派生结果提供基础概念 [1]。本文借用显式来源关联定位删除对象，不提出新的来源理论，也不假设来源图能完整表示模型的全部语义影响。

Garg、Goldwasser 和 Vasudevan 对被遗忘权背景下的数据删除进行了形式化研究 [2]。本文的逻辑存储检查比其讨论的广义删除保证更窄：我们检查选定数据库对象的内容和状态，而不声称对攻击者不可区分、历史备份或外部观察建立密码学保证。

CHESS 通过受控调度寻找和重现并发程序错误 [3]；有界黑盒崩溃测试通过有限工作负载和故障注入寻找一致性缺陷 [4]。本文采用可重复调度位置与明确终止屏障的思想，但没有枚举全部可达状态，也不提供模型检查意义上的完备性。五种通道和三个时序位置是预先声明的研究矩阵，并非真实环境的概率分布。

Yao 等提出执行状态遗忘，通过恢复与重放追求未观察目标信息的反事实轨迹 [5]。其讨论覆盖提示、压缩记忆与 KV cache，并依赖确定性转移、匹配的反事实观察和可恢复外部环境等条件。本文只研究持久存储删除，不测量模型行为，因此不能将数据库零残留解释为已实现该类执行状态遗忘。MemLineage 将签名来源与派生关联用于敏感动作的执行约束 [6]；其关联传播保证受已记录父边与归因阈值限制，研究目标与本文的删除完成条件不同。本文不宣称首次引入记忆来源、关联治理或删除与遗忘的区分。[5]、[6] 均按已核对的 arXiv v1 预印本引用，不将其表述为已同行评审的出版物。

3. 系统范围与完成条件

3.1 对象、存储与信任边界

研究对象为 Aervox-harness 的本地单用户 SQLite Memory 仓储切片。每个实验数据库包含待删除对象 `target` 和应保持不变的对象 `keep`。主表和修订表保留身份关系；清理将内容置空或将状态置为墓碑，而不是无差别删除所有行。目标涉及共享事实或其他生命周期拥有的派生投影时，生产组件拒绝直接完成，要求对应所有者先处理；本实验不绕过该边界。

逻辑存储	清理后的检查条件
主记录	恰有一条目标墓碑，正文为空，关键词和来源/当前修订/父引用清空，验证状态失效
历史修订	目标全部修订的正文为空
来源证据	关联目标全部修订的证据标记为墓碑，来源范围清空
FTS	不存在目标全文索引项
向量	不存在目标嵌入项
压缩摘要	摘要为空且标记处于墓碑状态
审计事件	目标事件的原因文本为空
Outbox	载荷只保留允许的身份字段及布尔删除标志，状态为已发布，错误文本为空

身份字段不被计作已擦除的内容。Outbox 的 `snapshotId` 仅允许字符串或空值，不允许携带嵌套内容对象。对 `keep`，观察器比较八类存储的完整行快照，而不仅检查记录数量。观察器还在数据库外保存已知 Outbox 事件的归属，防止错误载荷丢失 `memoryId` 后从实验观察范围消失。该额外归属是合成夹具的测量设施，不能被表述为生产数据库已有的独立来源账本。

3.2 指标定义

令 C 表示实验协议已宣告完成，R(t) 表示时刻 t 八类逻辑存储中的残留计数向量，K(t) 表示保留对象快照未改变。完成时观察记为 t_c，指定迟到写入后的最终观察记为 t_f。

- 错误完成：C 为真，但 R(t_c) 至少一项非零。
- 完成后复活：C 为真且 R(t_c) 全零，但 R(t_f) 至少一项非零。
- 安全完成：C 为真，R(t_c) 与 R(t_f) 均全零，且 K(t_f) 为真。
- 阻塞：验证器拒绝宣告完成。遗漏实验中的阻塞表示残留仍存在而完成请求被拒绝，不能计作成功删除。

这些指标是每个固定案例的判定规则。零观测失败只说明所列案例通过；它既不是未知调度的失败概率估计，也不是长期运行中永不复活的证明。

4. 删除机制与实现修复

4.1 清理、守卫与验证

生产 `clean()` 在写事务内检查对象是否受当前切片支持，随后更新目标墓碑、修订、证据、摘要和事件，清除 FTS 与向量，并将待处理压缩事件改为不含正文的终态载荷。正常派生写入通过仓储守卫重新确认目标仍可使用。FTS、向量、摘要、审计与 Outbox 通道的拒绝方式可以不同；实验根据数据库观察判断结果，不要求每个拒绝都以同一个异常返回。

`verify()` 使用新的数据库读取检查清理后的状态，不以 `clean()` 的返回值或影响行数代替验证。实验再使用独立 SQL oracle 检查八类存储，避免将生产验证器的判断直接作为实验真值。二者共享目标范围和墓碑合同，这是测量含义一致的必要条件；其查询实现分离并辅以各存储的残留正控制，以降低共同漏检风险。

上述事务不包含模型调用、网络 I/O 或工作者的全部生命周期。已获取的数据仍可留在工作者内存中；本研究要求其不能通过受检查的持久化入口写回已删除来源，不宣称抹除该工作者进程中的所有字节。

4.2 迟到错误回写

旧 Outbox 实现的 `markFailed` 与 `markDeadLetter` 根据事件 ID 更新状态及 `last_error`。后台压缩任务已经取得含正文的载荷后，即使清理将事件改为已发布墓碑，随后的异常仍可能把正文拼入错误信息，并将事件重新置为失败或死信。正常摘要写入上的来源守卫无法覆盖这条错误处理分支。

修复将三个条件置入同一条 SQL UPDATE：事件为 Memory 压缩事件时，其载荷未标记删除、对应来源记录存在、且来源未删除。SQLite 在实际更新时判断条件，避免把检查与写入分开为两个可交错步骤。其他类型事件继续遵循原失败与死信处理。回归同时放入无关的存活来源，验证守卫与每条事件自身的 `memoryId` 关联，避免被任意存活记录绕过。

4.3 完成验证的漏检修复

独立审查发现，旧验证器没有核对部分已经清空的来源引用、摘要墓碑状态和 Outbox 终态；对 Outbox 内容只检查 `summaryText`，无法发现其他键名下的残留。修复后，验证器逐项核对这些条件，要求删除标记为 JSON 布尔真，拒绝身份字段之外的载荷键和非字符串、非空的 `snapshotId`。清理时也将畸形的嵌套快照字段转换为空值。

针对这些问题的回归在干净状态上重新注入残留，验证拒绝完成，再执行清理重试并验证通过；它们不计入主实验的 80 个案例。补丁保持原证明标识 `memory-local-v2`，不会自动重验历史完成记录；历史证明的迁移与存量数据审计不在本次实验范围内。生产来源关联丢失或数据库被任意修改的情况也不能由该有界切片保证恢复。

5. 实验设计

5.1 消融与定序矩阵

四个消融臂使用相同夹具和预先声明的调度。`soft-delete` 仅更新来源删除标记；`cleanup-only` 清理内容但使用无守卫的研究写入；`guarded-cleanup` 使用生产派生写入守卫；`verified-guarded` 再加入生产完成验证。无守卫路径和遗漏路径是专门用于机制隔离的 SQL 变异，不是四个外部框架的实现，也不是新增的产品运行模式。

消融臂	标记及清理	派生写入守卫	完成前验证
soft-delete	仅标记	无	无
cleanup-only	清理	无	无
guarded-cleanup	清理	有	无
verified-guarded	清理	有	有

对五个派生通道，分别将一次迟到写入安排在清理前、清理后但宣告完成前、宣告完成后，共 15 个案例。另在五个案例中每次故意遗漏一个通道的清理。每臂 20 个案例各执行一次，总分母为 80。迟到正文来自删除前已取出的同一份载荷；不通过生成不同内容改变各臂难度。每个案例使用独立临时数据库，正常清理不访问用户实际数据库。

这些调度是按程序步骤主动安排的可重复交错位置，不依赖线程竞争偶然命中，也不是同时启动大量随机工作者。案例中清理和写入的相对顺序可解释，但选定三处位置无法涵盖事务内部的全部指令交错、真实多连接争用或其他业务工作者。

5.2 独立观察器与有效性正控制

独立 oracle 直接读取表和索引，不调用生产 `verify()`。它在宣告完成处及最终写入后各观察一次，并比较 `keep` 快照。观察器不能因为查询为空就默认成功：协议明确要求目标主记录仍有一条被清空的墓碑。证据查询沿目标全部修订关联，不能绑定夹具中某一条修订的命名。

我们在正常清理后逐一向八类存储注入残留，确认 oracle 能识别对应通道；额外测试非初始修订、失去来源字段的已知 Outbox 事件、畸形 JSON、扩展内容字段和嵌套快照对象。还通过破坏 `keep` 的额外证据确认快照比较能发现误清理。上述正控制用于测试测量工具自身的敏感性，不将其数量添加到论文主表，以免混淆实验单元。

5.3 耗时协议

每臂重复 24 次 `fts:before-clean`，共 96 条独立记录的计时样本。每次使用从初始化 schema 克隆的新文件和连接，Node 进程及模块已加载；每轮轮换四个消融臂的执行顺序。计时只覆盖清理与可选验证，排除建库、夹具构造、迟到写入和独立 oracle。计时样本同时保存完成标志与最终观察，防止把未完成或仍有残留的清理误计为成功耗时；只标记的基准臂按设计保留其不完整删除语义。

报告给出第 25 百分位、中位数、第 75 百分位与第 95 百分位。分位数使用排序样本上索引 `(n-1)*p` 的线性插值，即 Hyndman-Fan type 7。我们不对这些本机测量进行显著性推断，不把四臂微基准差异解释为真实应用吞吐量，也不宣称进程或磁盘缓存处于冷启动状态。

5.4 错误回写与终止恢复

错误回写探针对 `markFailed` 和 `markDeadLetter` 分别比较忠实重建的旧 UPDATE 与补丁后的生产方法，共四个案例。每个案例先正常清理并验证，再从已缓存的敏感载荷构造错误文本，观察其是否回到数据库。

终止恢复包含两个独立文件数据库探针。第一处在子进程执行两条手工 SQL、但写事务尚未提交时发送 SIGKILL，重新打开后比较目标完整快照以确认回滚；它是代表性的 SQLite 事务探针，不是在生产 `clean()` 内部插入终止点。第二处在生产 `clean()` 已提交后终止子进程，重新打开后检查清理状态。两者最后都再次调用清理与验证，检查幂等重试及 `keep` 完整性。SIGKILL 终止进程，不模拟操作系统断电、磁盘固件行为或任意 WAL 损坏。

5.5 数据与源文件溯源

实验报告记录基线提交 `99c82924`、测量时 HEAD、工作区状态、Node/平台/CPU，以及 56 项关键源码、执行 JavaScript、schema、实验脚本和工具链文件的 SHA-256。因为工作区可能包含尚未提交的补丁，只报告 HEAD 不足以标识实际被测程序。测量结束后重新计算同一文件集合的哈希；若出现漂移或读取错误，先保存原始 JSON/CSV，再以失败状态退出。

报告校验器重新计算案例分类、总数、分位数和恢复判定，验证完整矩阵以及 CSV 与 JSON 一致。源码校验是另一项显式检查：旧实验的归档报告可以保留，但不能在源码变化后继续作为当前实现的测量结果。耗时结果依赖环境，不要求不同机器复现报告逐字节一致。

6. 实验结果

6.1 删除完成与复活

消融臂	案例数	宣告完成	错误完成	完成后复活	安全完成	正确阻塞
soft-delete	20	20	20	0	0	0
cleanup-only	20	20	10	5	5	0
guarded-cleanup	20	20	5	0	15	0
verified-guarded	20	15	0	0	15	5

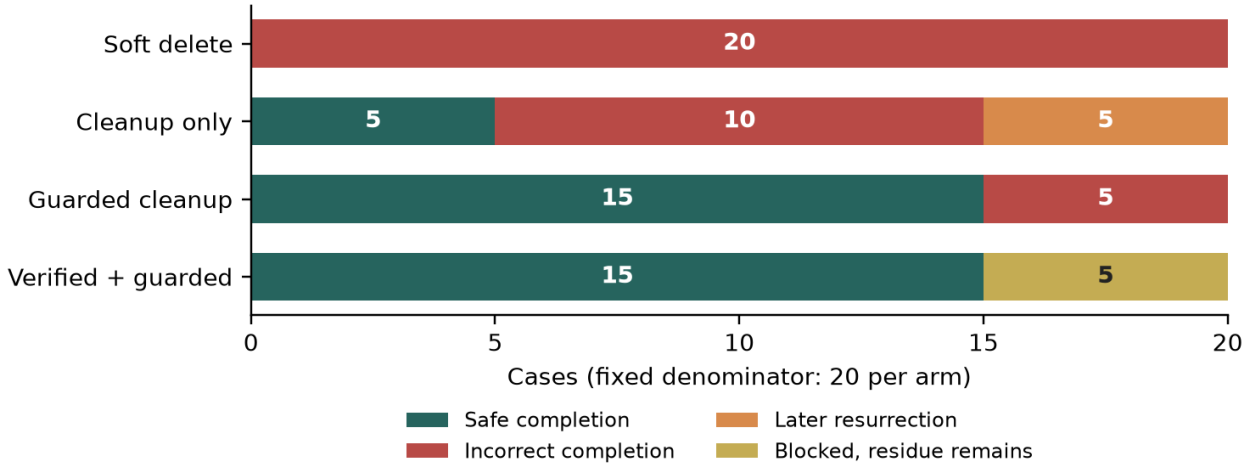


图 1：四个消融臂在固定 20 个案例上的结果分类。阻塞案例仍有残留，未计为成功删除。

仅标记的消融臂无法清空正文及派生内容，因此 20 次均错误完成。只清理的消融臂在写入发生于清理前时可以清除已有结果；清理后、完成前的五次写入造成完成时残留，五种遗漏也造成残留，合计 10 次错误完成。发生于完成后的五次写入则表现为先干净、后复活。

生产写入守卫阻止所选迟到结果持久化，使 `guarded-cleanup` 的五次完成后复活消失，但它无法发现某个清理通道根本没有执行；因此仍有五次遗漏对应错误完成。加入验证后，这五次请求全部被拒绝，另 15 次安全完成。最终仍有残留的案例数依次为 20、15、5、5；后两臂的差别主要是能否诚实拒绝完成，而不是验证器自行删除了遗漏内容。所有主矩阵案例中 `keep` 均保持不变，没有未处理实验异常。

该结果回答 RQ1：在所选矩阵内，写入守卫针对未来回写，完成验证针对当前残留，二者解决不同问题。只增加其中一种不能替代另一种。

6.2 错误回写与恢复结果

方法或终止边界	观察结果
旧 markFailed	清理后错误文本重新进入 Outbox，状态变为 failed
修复后 markFailed	无内容复活，事件保持 published
旧 markDeadLetter	清理后错误文本重新进入 Outbox，状态变为 dead_letter
修复后 markDeadLetter	无内容复活，事件保持 published
手工写事务未提交时 SIGKILL	重开后目标完整快照与终止前一致，观察到回滚；重试后干净
生产清理提交后 SIGKILL	重开后已清理状态保留；再次清理与验证通过

四个错误回写案例中，旧实现的两个案例均复活，补丁后的两个案例均未复活，回答 RQ2。此处的 2/2 是两种明确方法的覆盖，不是统计风险概率。两处终止探针均保留未删除对象，回答 RQ3 在这两个边界上的结果；不能据此推导所有崩溃位置都正确。

6.3 操作耗时

测量环境为 Apple M5、macOS arm64、Node v24.21.0；报告生成时间为 2026 年 10 月 15 日 20:02:17（Asia/Shanghai）。每臂 n=24，表中单位为毫秒，显示值四舍五入到小数点后三位；原始精度保留在 JSON 中。

消融臂	P25	中位数	P75	P95
soft-delete	0.110	0.218	0.425	0.580
cleanup-only	0.854	1.157	1.502	2.138
guarded-cleanup	0.836	1.202	1.738	2.617
verified-guarded	2.613	3.425	4.288	6.263

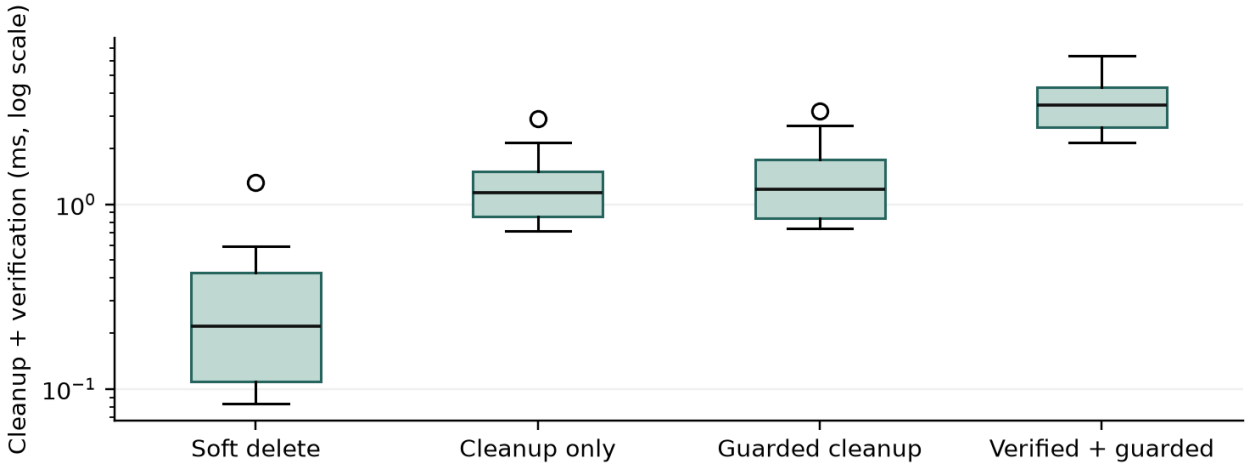


图 2：每臂 24 个清理加验证耗时样本。纵轴采用对数刻度，箱体与离群点用于描述样本分布。

耗时分布显示完整机制需要额外的数据库观察；对本文的两记录夹具，清理与验证均处于毫秒或亚毫秒至毫秒的操作尺度。四臂的精确分位数、CPU 型号及 Node 版本列于随文实验报告。该成本不包含数据库初始化和模型调用，不能外推为大型记忆库的删除延迟或真实产品的端到端响应时间。重复样本可能受文件系统缓存、后台负载和执行顺序影响，因此不使用“几乎零成本”或“显著加速”等描述。

7. 讨论与有效性边界

本研究最直接的工程启示是把含内容的错误记录视为删除范围的一部分。只审查正常结果表会漏掉失败重试、死信与日志式字段。检查是否允许写入也应与实际 UPDATE 共享原子边界；预先读取的载荷和状态不能替代当前来源状态。另一方面，守卫并不证明此前所有清理步骤成功，完成前仍需核对持久存储。

内部有效性。 实验 oracle 与生产验证器虽然实现分离，仍由同一研究任务定义存储范围，可能共享遗漏的对象模型。各通道的残留正控制和额外修订/载荷变异降低了已知共同漏检，但不能证明未知字段与未纳入的派生物不存在。夹具使用完整保留快照检查误清理，且所有遗漏被预先列入协议，避免仅展示成功路径。

外部有效性。 两条合成记录不能代表大数据库、复杂关联图、多用户、跨设备同步或多进程高争用。本研究没有比较外部记忆框架、不同 SQLite 引擎或不同硬件，也没有运行真实模型。实际共享投影需要其他所有者处理，不能直接从本文推导整个产品的删除覆盖已经完成。

时序与故障范围。 三个写入位置是定序实验，不是完整并发状态空间。两个 SIGKILL 屏障只检查指定提交边界，第一处没有进入生产清理函数内部。事务、文件系统与磁盘缓存之间的断电持久性，以及进程重启后所有工作者的再次调度，都需要额外实验。

保证层级。 本文的“干净”表示逻辑查询范围内内容被清空或相应派生行不存在。SQLite 页、WAL、文件系统快照、备份、进程内存及模型权重没有接受擦除验收。保留身份元数据的墓碑也不是数据库中完全没有任何目标痕迹。语义影响和对外已执行动作不属于本文判定函数。

历史状态。 当前补丁改善未来执行的清理和验证，并不撤销历史 `memory-local-v2` 证明，也未对真实用户库开展扫描、迁移或重验。论文完成、实验复现、CI 通过、PR 审查以及正式发布是不同状态，不能相互替代。

8. 结论

针对本地 SQLite 的异步 Memory 派生物，本文通过四臂定序消融区分了完成时残留和完成后复活。所选案例表明，原子写入守卫消除了受测迟到回写，独立完成验证拒绝了有意遗漏的清理，两者共同形成 15 次安全完成与 5 次正确阻塞。错误回写探针发现了生产处理路径的实际缺陷，增强后的验证及观察器减少了已知假阴性，两个终止边界支持该范围内的恢复与重试。后续工作应在来源关系完整性、共享派生物、多工作者调度和规模扩展方面建立新的证据，再讨论更广泛的删除或语义遗忘保证。

参考文献

[1] Peter Buneman, Sanjeev Khanna, Wang-Chiew Tan. Why and Where: A Characterization of Data Provenance. ICDT 2001, LNCS 1973, pp. 316-330. Springer, 2001. DOI: 10.1007/3-540-44503-X_20.

[2] Sanjam Garg, Shafi Goldwasser, Prashant Nalini Vasudevan. Formalizing Data Deletion in the Context of the Right to be Forgotten. Cryptology ePrint Archive, Paper 2020/254, 2020. 本文引用 ePrint 修订版。原文。

[3] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, Iulian Neamtiu. Finding and Reproducing Heisenbugs in Concurrent Programs. 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08), 2008. 会议记录。

[4] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnapalli, Pandian Raju, Vijay Chidambaram. Finding Crash-Consistency Bugs with Bounded Black-Box Crash Testing. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pp. 33-50, 2018. 会议记录。

[5] Chao Yao, Yangbo Wei, Zhen Huang, Junhong Qian, Chenle Chen, Shaoqiang Lu, Chen Wu, Lei He. Forgetting Without Restarting: Execution-State Unlearning for Stateful LLM Agents. arXiv:2609.04875v1, 2026. 预印本，版本 1。原文。

[6] Ciyan Ouyang, Rui Hou. MemLineage: Lineage-Guided Enforcement for LLM Agent Memory. arXiv:2605.14421v1, 2026. 预印本，版本 1。原文。

附录 A：复现与结果检查

完整入口为同目录 README，文献元数据见 `references.bib`。以下命令从 `feat/memory-deletion-paper` 的仓库根目录执行；依赖由根目录统一安装，Node/pnpm 使用 `mise.toml`。所有实验使用新临时目录，不访问生产数据。

```
mise exec -- pnpm install --frozen-lockfile
mise exec -- pnpm exec turbo run build --filter=@aervox/repositories...
mise exec -- node --test scripts/memory-deletion-research/runner.test.mjs
study_run="$(mktemp -d /tmp/aervox-memory-deletion.XXXXXX)"
mise exec -- node scripts/memory-deletion-research/runner.mjs \
  --out="$study_run" --repetitions=24
mise exec -- node scripts/memory-deletion-research/validate-report.mjs \
  "$study_run" --check-source
```

预期正确性汇总为第 6 节所列四臂结果，错误回写为旧实现 2 次复活、修复后 0 次，两个恢复探针均通过；耗时无需与归档值完全相等。输出目录已有报告时脚本默认拒绝覆盖。任何意外案例或源码漂移都应查看已保存的原始证据，不通过修改汇总数值令校验器通过。

生成图表与 PDF 的 Python 依赖锁定于研究脚本目录的 `requirements.txt`。用该环境运行 `plot-results.py` 和 `render-paper.py`，并对 PDF 逐页视觉检查。修改研究源码后应重新实验、同步报告和图表，再更新本文数值；纯文字修订可以复用来源校验通过的实验快照。`pnpm check:memory-deletion` 将研究回归和归档一致性纳入代码门禁，`./aervox ci all` 执行仓库全量终审。